

Core Attributes & Usage(Linear Layout)

1. What is the default orientation?

The default orientation is **horizontal**. If you omit the android:orientation attribute, views will align side-by-side.

2. How does android:layout_weight work?

It distributes leftover screen space among child views proportionally. To ensure calculations work reliably, set the target dimension (android:layout_width for horizontal or android:layout_height for vertical layouts) to 0dp.

3. What is android:weightSum used for?

It defines the maximum weight total for the layout. For example, if you set weightSum="100" and a child view has layout_weight="20", that child will occupy exactly 20% of the available space.

4. What is the difference between gravity and layout_gravity?

android:gravity controls how the contents *inside* the layout or view are aligned. android:layout_gravity tells the child view how to position *itself* inside its parent container.

5. Compare the different Layouts

LinearLayout vs. Other Layouts

Feature	LinearLayout	RelativeLayout	ConstraintLayout
Arrangement	Single line (row/column)	Relative parent/siblings	to Flexible constraint links
Nesting Needed?	High (for complex structures)	Moderate	Low (flattens view hierarchy)
Performance Pass	Single pass (Double if weights used)	Double layout pass	Optimized single pass

6. What are the Performance Pitfalls (Double Taxation) of Linear Layout? Explain.

Nesting multiple LinearLayout view groups inside each other to build a complex UI causes "**double taxation**". When you assign layout_weight to a child, the system is forced to run a double layout-and-measure pass over the hierarchy to calculate exact boundaries. To optimize performance and avoid laggy rendering, modern Android guidelines suggest replacing complex nested layouts with a flat ConstraintLayout on Android Developers.